IOT– Fire Fighting Robot

Submitted By –

1. Arpit Rajendra Shirbhate	24BCE7582
2. Akash Rajeev Shrivastava	24BCE7650
3. Nithin B	24BCE7427
4. Saurish Chandra	24BCE7664
5. Smruti Ranjan Giri	24BCE7527
6. Sumedh Singh Gautam	24BCE7071

Under Guidance of –

Prof. Madhusmita Mohanty

College Details-

VIT-AP UNIVERSITY, G-30, INAVOLU, AMARAVATHI – 522241
ANDHRA PRADESH.

TOPICS	Page Number
Introduction	02
Background	03
Problem Definition	04
Objectives	05
Methodology	06
Results and Discussion	07
Conclusion and Future Scope	08
References	09
Arduino Code	10

IOT Based Fire- Extinguisher Robot

❖ Introduction

The IoT-Based Fire Fighting Robot is an innovative system designed to detect and extinguish fires automatically without human intervention. The robot utilizes an Arduino Uno microcontroller integrated with flame sensors, a water pump, and DC motors to locate and suppress fire sources effectively. Upon detecting a fire, the flame sensor sends a signal to the controller, which drives the robot toward the fire and activates the water pump for extinguishing.

The system incorporates IoT (Internet of Things) technology. This enhances safety and allows users to track fire incidents from any location. The project aims to minimize fire-related damages and reduce risks faced by human firefighters, particularly in hazardous or hard-to-reach environments.

The prototype demonstrates accurate fire detection within a short response time and successful extinguishing in small-scale fire scenarios. It serves as a foundation for future advancements in autonomous firefighting robots, with potential upgrades including AI-based navigation, smoke and temperature sensors, and GPS-enabled outdoor operation.

❖ Background

Fire accidents have always been one of the most devastating hazards, leading to loss of property, infrastructure, and human life. In industrial environments, residential buildings, and remote areas, such incidents can escalate rapidly if not controlled in time. Traditional fire detection systems, such as smoke detectors and alarm systems, are limited in functionality — they can only alert occupants or authorities about the presence of fire but cannot take direct action to suppress it. This delay between detection and response often results in severe consequences.

In recent years, with advancements in embedded systems and IoT (Internet of Things) technologies, the concept of **autonomous fire-fighting robots** has gained significant attention. These robots combine sensors, actuators, and microcontrollers to detect, locate, and extinguish fires without requiring human intervention. Such systems can operate in environments that are too dangerous for humans, such as chemical plants, tunnels, and disaster zones.

The integration of IoT technology allows these robots to transmit real-time data to users or control centers, enabling remote monitoring, data logging, and emergency alerts. This connectivity ensures that fire incidents can be managed more effectively, even from a distance, thereby improving safety and response time.

❖ Identification of Problem

Fire accidents can cause severe damage and loss of life, especially in unmanned or hazardous areas where human intervention is delayed. Traditional detection systems such as smoke alarms and sprinklers can only alert occupants but cannot take immediate corrective action. This delay between detection and response often results in significant destruction and risk to human lives. Therefore, there is a pressing need for an autonomous IoT-enabled robot that can not only detect fire but also respond intelligently by avoiding obstacles, extinguishing flames, and alerting users remotely for faster action and improved safety.

In many industrial and residential environments, fires can occur in areas that are either inaccessible or unsafe for human firefighters. For example, chemical factories, mining zones, and nuclear plants pose extreme risks due to high temperatures, toxic gases, and explosive materials. In such environments, deploying human responders is dangerous and sometimes impossible. An autonomous fire-fighting robot can operate in these conditions safely, continuously monitoring for fire hazards and taking prompt action before the situation escalates.

❖ Objectives of the Proposed Work

1. To design and develop a robot that can detect and extinguish fire automatically.
2. To use IoT technology for monitoring the robot remotely.
3. To ensure quick response during fire accidents and reduce human risk.
4. To build a cost-effective and reliable system for small-scale fire emergencies.
5. To test the robot's performance in detecting and extinguishing fire efficiently.

❖ Plan of Action

1. **Requirement Analysis**
   - Identify necessary hardware: Arduino Uno, flame sensors, HC-SR04 ultrasonic sensor, L298N motor driver, water pump with relay, chassis with motors, power supply.
   - Define fire detection threshold and obstacle detection range.

2. **Hardware Assembly**
   - Mount flame sensors at front angles for wide detection coverage.
   - Install ultrasonic sensor on front (or servo for scanning).
   - Connect L298N to motors and Arduino.
   - Wire relay to control water pump.
   - Connect Motor (Submersible) for water spray.

3. **Software Development**
   - Write Arduino code for:
   - Fire detection and triggering water pump.
   - Obstacle detection and avoidance logic.
   - GSM alert message sending.
   - Autonomous movement control.
   - Test each module individually before integration.

4. **Integration & Testing**
   - Combine all modules into a single program.
   - Test in controlled environment with simulated fire and obstacles.
   - Adjust sensor thresholds and motor speeds for optimal performance.

5. **Deployment & Documentation**
   - Conduct final demonstrations in different scenarios.
   - Record performance data (reaction time, detection accuracy, obstacle handling).
   - Prepare final report with problem statement, objectives, methodology, and resul

❖ Results and Discussion

1. The IoT-based fire fighting robot was successfully designed and implemented using Arduino Uno as the main controller. The system was able to detect the presence of fire through the flame sensor and respond immediately by activating the water pump to extinguish the fire. The robot's movement was controlled through DC motors, which allowed it to navigate toward the fire source effectively.

2. The integration of IoT allowed remote monitoring and control of the robot using a mobile or web interface. This feature ensured that the robot could be operated even from a safe distance, reducing the risk to human life in case of emergencies.

3. During testing, the robot performed well in detecting small flames and putting them out in a short amount of time. The response time was fast, and the sensors were accurate in identifying fire within their detection range. However, the robot's performance was affected slightly on uneven surfaces or in areas with strong light interference.

Overall, the project demonstrated that IoT-based fire fighting systems can be an effective and low-cost solution for early fire detection and control, especially in small industrial areas, offices, and homes.

❖ Conclusion and Future Scope

1. Conclusion

- The IoT-based Fire Fighting Robot project successfully demonstrated an automated approach to detecting and extinguishing fires using sensor technology and Arduino control. The system efficiently detected flames and responded by activating the water pump to suppress the fire, while IoT connectivity allowed for remote monitoring and control. This project highlights the potential of combining robotics and IoT to enhance safety in environments prone to fire hazards. It offers a reliable and cost-effective solution for early fire detection and prevention in small-scale applications.

2. Future Scope

- In the future, the robot can be enhanced by integrating advanced technologies such as artificial intelligence (AI) and machine learning (ML) for smarter decision-making and improved path navigation. The use of thermal cameras and GPS modules can allow the robot to locate fire sources more precisely in larger or complex areas. Additionally, using stronger materials and improved motor systems can make the robot suitable for industrial use. The IoT system can also be expanded to send real-time alerts to emergency services and provide live video feeds for better situational awareness.

❖ References

1. Arduino Official Documentation — https://www.arduino.cc
2. Espressif IoT Development Framework — https://www.espressif.com
3. “Design and Implementation of Fire Fighting Robot” — IEEE Paper, 2020.
4. Arduino Based Fire Fighting Robot, IJIRMPS
5. ResearchGate, “IoT Based Fire Fighting Robot Using Arduino Uno,” 2021.

Appendix

```
#include <Servo.h>

// Define flame sensor pins
#define FLAME_SENSOR_LEFT A0
#define FLAME_SENSOR_CENTER A1
#define FLAME_SENSOR_RIGHT A2

// Define motor driver pins
#define MOTOR_L1 5  // Left Motor Forward
#define MOTOR_L2 6  // Left Motor Backward
#define MOTOR_R1 9  // Right Motor Forward
#define MOTOR_R2 10 // Right Motor Backward

// Define relay for water pump
#define RELAY_PIN 7

// Define servo motor pin
#define SERVO_PIN 3

// Create Servo object
Servo waterServo;

// Flame sensor threshold
#define FIRE_DETECTED_THRESHOLD 800  // Adjust based on sensor readings
```

```
void setup() {
    // Initialize Serial Monitor
    Serial.begin(9600);

    // Set motor pins as output
    pinMode(MOTOR_L1, OUTPUT);
    pinMode(MOTOR_L2, OUTPUT);
    pinMode(MOTOR_R1, OUTPUT);
    pinMode(MOTOR_R2, OUTPUT);

    // Set relay as output
    pinMode(RELAY_PIN, OUTPUT);
    digitalWrite(RELAY_PIN, LOW);  // Ensure relay (pump) is OFF at startup

    // Initialize servo motor
    waterServo.attach(SERVO_PIN);
    waterServo.write(90); // Set initial servo position to center

    // Set flame sensor pins as input
    pinMode(FLAME_SENSOR_LEFT, INPUT);
    pinMode(FLAME_SENSOR_CENTER, INPUT);
    pinMode(FLAME_SENSOR_RIGHT, INPUT);

    Serial.println("🚀 Fire Fighting Robot Initialized...");
}
```

```
void loop() {
    // Read flame sensor values
    int flameLeft = analogRead(FLAME_SENSOR_LEFT);
    int flameCenter = analogRead(FLAME_SENSOR_CENTER);
    int flameRight = analogRead(FLAME_SENSOR_RIGHT);

    // Print sensor values to Serial Monitor
    Serial.print("Flame Left: ");
    Serial.print(flameLeft);
    Serial.print(" | Flame Center: ");
    Serial.print(flameCenter);
    Serial.print(" | Flame Right: ");
    Serial.println(flameRight);

    // Check flame detection
    if (flameCenter < FIRE_DETECTED_THRESHOLD) {
        Serial.println("🔥 Fire Detected at Center! Moving Forward...");
        moveForward();
        activateWaterPump();
    }
    else if (flameLeft < FIRE_DETECTED_THRESHOLD) {
        Serial.println("🔥 Fire Detected at Left! Turning Left...");
        turnLeft();
        activateWaterPump();
    }
    else if (flameRight < FIRE_DETECTED_THRESHOLD) {
        Serial.println("🔥 Fire Detected at Right! Turning Right...");
        turnRight();
        activateWaterPump();
    }
    else {
        Serial.println("✅ No Fire Detected. Stopping...");
        stopMotors();
        deactivateWaterPump();  // Ensure pump is OFF when no fire is detected
    }

    delay(500); // Delay for stability
}
```

```
// Function to move forward
void moveForward() {
    digitalWrite(MOTOR_L1, HIGH);
    digitalWrite(MOTOR_L2, LOW);
    digitalWrite(MOTOR_R1, HIGH);
    digitalWrite(MOTOR_R2, LOW);
    delay(350);
    stopMotors();
}

// Function to turn left
void turnLeft() {
    digitalWrite(MOTOR_L1, LOW);
    digitalWrite(MOTOR_L2, HIGH);
    digitalWrite(MOTOR_R1, HIGH);
    digitalWrite(MOTOR_R2, LOW);
    delay(500);  // Adjust timing
    stopMotors();
}

// Function to turn right
void turnRight() {
    digitalWrite(MOTOR_L1, HIGH);
    digitalWrite(MOTOR_L2, LOW);
    digitalWrite(MOTOR_R1, LOW);
    digitalWrite(MOTOR_R2, HIGH);
    delay(500);  // Adjust timing
    stopMotors();
}

// Function to stop motors
void stopMotors() {
    digitalWrite(MOTOR_L1, LOW);
    digitalWrite(MOTOR_L2, LOW);
    digitalWrite(MOTOR_R1, LOW);
    digitalWrite(MOTOR_R2, LOW);
}
```

```
// Function to activate water pump and move servo
void activateWaterPump() {
    Serial.println("🚰 Activating Water Pump...");
    digitalWrite(RELAY_PIN, HIGH);  // Turn ON the water pump

    // Move the servo from left to right
    for (int pos = 60; pos <= 120; pos += 5) {
        waterServo.write(pos);
        delay(100);
    }

    // Move the servo from right to left
    for (int pos = 120; pos >= 60; pos -= 5) {
        waterServo.write(pos);
        delay(100);
    }

    delay(2000);  // Pump stays on for 2 seconds
    deactivateWaterPump();  // Turn off the water pump
}

// Function to deactivate water pump
void deactivateWaterPump() {
    Serial.println("✅ Deactivating Water Pump...");
    digitalWrite(RELAY_PIN, LOW);  // Turn OFF the water pump
    waterServo.write(90);  // Reset servo to center position
}
```